

Design and Evaluation of an AI-Assisted Grading Tool for Introductory Programming Assignments: An Experience Report

Goda Nagakalyani
nagakalyani@cse.iitb.ac.in
Indian Institute of Technology,
Bombay
Mumbai, India

Saurav Chaudhary
saurav@cse.iitb.ac.in
Indian Institute of Technology,
Bombay
Mumbai, India

Varsha Apte
varsha@cse.iitb.ac.in
Indian Institute of Technology,
Bombay
Mumbai, India

Ganesh Ramakrishnan
ganesh@cse.iitb.ac.in
Indian Institute of Technology,
Bombay
Mumbai, India

Srikanth Tamilselvam
srikanth.tamilselvam@in.ibm.com
IBM India Research Labs
Bengaluru, Karnataka, India

Abstract

In a typical introductory programming course, grading student-submitted programs involves an autograder which compiles and runs the programs and tests their functionality with predefined test cases, with no attention to the source code. However, in an educational setting, grading based on inspection of the source code is required for two main reasons (1) awarding partial marks to ‘partially correct’ code that may be failing the testcase check (2) awarding marks (or penalties) based on source code quality or specific *criteria* that the instructor may have laid out in the problem statement (e.g. ‘implement sorting using bubble-sort’). However, grading based on studying the source code can be highly time consuming when the course has a large enrollment. In this paper we present the design and evaluation of an *AI Assistant* for source code grading, which we have named *TA Buddy*. *TA Buddy* is powered by Code Llama, a large language model especially trained for code related tasks, which we fine-tuned using a graded programs dataset. Given a problem statement, student code submissions and a *grading rubric*, *TA Buddy* can be asked to generate *suggested grades*, i.e. ratings for the various rubric criteria, for each submission. The human teaching assistant (TA) can then accept or overrule these grades. We evaluated the *TA Buddy*-assisted manual grading against ‘pure’ manual grading and found that the time taken to grade reduced by 24% while maintaining grade agreement in the two cases at 90%.

CCS Concepts

• **Applied computing** → **Computer-managed instruction**; • **Computing methodologies** → **Natural language processing**.

This research was supported partly by Tower Research Capital India and partly by IBM.



This work is licensed under a Creative Commons Attribution-ShareAlike International 4.0 License.

SIGCSE TS 2025, February 26-March 1, 2025, Pittsburgh, PA, USA
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0531-1/25/02
<https://doi.org/10.1145/3641554.3701913>

Keywords

Source Code Evaluation, AI-Assisted Grading, LLMs, Programming Assignments, CS Education, Rubric, Grading

ACM Reference Format:

Goda Nagakalyani, Saurav Chaudhary, Varsha Apte, Ganesh Ramakrishnan, and Srikanth Tamilselvam. 2025. Design and Evaluation of an AI-Assisted Grading Tool for Introductory Programming Assignments: An Experience Report. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE TS 2025)*, February 26-March 1, 2025, Pittsburgh, PA, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3641554.3701913>

1 Introduction

Computer Programming has been a highly sought-after skill for many decades now, and remains highly popular even after the emergence of generative AI. Enrollment can range from around 1000 (in colleges) to more than 10000 (in online MOOCs [16]).

The best way to assess the programming skill of students is to give them ‘hands-on’ programming *lab exercises*, where they write the program on a computer to implement a given functionality, which can be compiled and run on some testcases [4]. E.g. an introductory programming course (henceforth referred to as ‘CS 101’) in our own engineering college IIT Bombay conducts a weekly programming *lab* for all enrolled students. A course with an enrollment of 1000 students, 12 labs conducted in the semester, with about 6 problems each, will result in 72,000 submissions to be graded. This number is of course much higher in case of MOOCs. Not surprisingly, auto-grading of programs has become almost a norm, and a large number of programming autograders with slightly varying features are available for instructors to use [1, 11, 13, 35]. It is important to note however that most programming autograders only check for *functionality*, and this check is usually done using a *fixed* set of input-output *testcases*.

In an educational setting, however, such a check can be unsatisfactory. For students, it is demotivating to be given a zero for a submitted program, say because of a small output formatting error. On the other hand, some student’s submitted programs may get undeserved marks by passing all testcases even though it is grossly violating some ‘requirements’ given in the problem statement (e.g. see Figure 1). Thus *manual grading* based on *code inspection*, carried

out by Teaching Assistants (TAs) is necessary. In our college, IIT Bombay, *all* CS 101 course instructors in at least the last 20 offerings required TAs to award final marks based on such code inspection, even though a programming autograder has been available for this duration.

Such manual grading is quite a human-resource intensive task since the TAs have to check for small errors, and very specific and uncommon code compliance ‘criteria’. One of our studies (reported in Section 6) shows that when clear grading rubrics are specified, then on average, a submission can be ‘manually graded’ by an experienced TA in 2.7 minutes. If only 4 of the 12 labs mentioned above are graded in such a manner, the time required will be 135 person days assuming an 8-hour day. Grading should be done in approximately 1 day for each of the graded labs, so about 34 *experienced* grader TAs will be required. These calculations can quickly reach ‘infeasible’ levels for larger enrollments.

To address these scale issues, efforts have been made to automate source code evaluation. Techniques such as *Static analysis* of code [3, 8, 18] have been used to evaluate source code. Static code analysis tools check the semantic difference between the student code and the instructor reference solution. However these do not work well for even slightly larger source codes [18]. Various solutions based on *Machine Learning (ML)* models have also been proposed [14, 19], which evaluate code based on presence of certain features or similarity to reference solutions. Existing ML methods are restricted to grading code only on generic features (e.g. is the student using ‘advanced’ programming constructs [29]), and are not easy to generalize to be able to grade on very problem-specific criteria. Recently, pre-trained Large Language Models (LLMs) have demonstrated immense potential in program comprehension tasks such as code summarization or code explanation [5, 27] for almost *any* code. Thus LLMs seem most suitable for developing a highly generalized AI model for grading a variety of different source codes, each of which may have unique grading criteria set by the instructor.

In this work, we present the design and evaluation of an AI-assisted grading feature, TA-Buddy, that we have incorporated into our own existing Learning Management System, Evalpro integrated in the Bodhitree interface[7]. Evalpro currently supports conventional test-case-based auto-grading, along with the ability for an instructor to specify a manual grading rubric, which consists of various criteria and ratings for each criterion. Grader TAs grade in a *fully manual* manner by manually selecting ratings for the source code and optionally writing a comment explaining their ratings. Our new feature aims to enhance this experience by allowing the instructor to first carry out an *autograde* run, where the AI-powered **TA-Buddy** auto-evaluates the source code with reference to the rubric and presents to the Grader TA *suggested ratings* for each criterion of the rubric, along with reasoning for that rating. The Grader TA can then either select the same rating and reasoning or select a different rating and write their own reasoning. They can also select the suggested rating but overwrite the reasoning.

The AI model that powers the TA-Buddy is *Code Llama-7b-Instruct* model [26] fine-tuned with a dataset of graded CS 101 programs. The effectiveness of TA-Buddy was evaluated by comparing fully manual grading with TA-Buddy-assisted grading of the same lab problems.

Sorting strings

Write a C++ program to sort n character strings. The strings need not be of equal length, but the maximum length of any string should be 11. The strings should be stored in an appropriate 2D array.

You need to write **string-compare** and **string-sort** functions. Sorting should be done using the bubble-sort algorithm. You are not allowed to implement the sorting within main.

Figure 1: Example of a problem statement containing specific code requirements given by the instructor.

Our study shows that manual grading with TA-Buddy saved time (24% on average), and also resulted in increased marks to students (16% on average), since programs failing on testcases also received partial marks.

The contributions of this work are summarized as follows:

- (1) We integrated an source code grader (termed as ‘TA Buddy’) based on a fine-tuned Large Language Model into our existing Bodhitree-Evalpro interface, which generates *suggested ratings* for all the criteria of a program evaluation.
- (2) We created a training dataset based on real CS101 submissions, their grading rubrics, and the grades allocated by TAs.
- (3) We evaluated our LLM-powered TA Buddy by conducting an AI-assisted as well as ‘purely Manual’ grading exercise on the same set of 6 lab problems with 740 submissions in all. We report the results that compare time saved using AI-assisted grading with purely manual grading; we also study how close AI-assisted grades are to ‘pure’ manual grades.

The paper is organized as follows: in Section 2 we present some background on source code grading, which we used in our own college introductory programming course CS 101; in Section 3 we discuss related work in source code based autograding; in Section 4 the workflow of AI-Assisted Grading using TA-Buddy is discussed; in Section 5 we present the details of how the training dataset was obtained; in Section 6 we will present the experiment done to evaluate the effectiveness of the AI-Assisted grading. We conclude the paper in Section 7 with a discussion of the impact and some limitations of the work presented here and future research directions.

2 Background: Grading of Source Code of CS101 Programs

The main objective of an introductory programming course (‘CS 101’) is for students to learn to develop working programs. Weekly programming assignments are used for this purpose. The assessment of programming assignments in CS 101 is generally done on autograders which compile and run the program and check their functionality using some specific input-output test cases [11]. Typically, for a graded lab, some testcases are *visible*, to help the student understand what the program is supposed to do, while some are *hidden* which are the ones used to award marks. However, in addition to this basic functionality specification of a programming assignment, instructors often specify numerous guidelines about the code itself, which the students are required to follow. Figure 1 shows a sample problem statement from an actual programming course in our institute, that includes specific instructions about

Table 1: Rubric for the problem *Sorting strings*, as an example. Note that the ratings across different criteria are not comparable except for the fact that the rating k is meant to be strictly greater than rating $k+1$ for all values of k , for each criterion.

Criteria	Rating-1	Rating-2	Rating-3
Quality of variable names	Good Variable Names	Ok Variable Names	Poor Naming of variables.
Sort is implemented in function <code>string_sort</code> .	Sorting logic implemented in separate function.	Sorting logic implemented in main function.	No logic for implementation of sorting.
Efficiency of Bubble sort algorithm	Sorting algorithm implemented correctly.	Sorting algorithm implemented is partially correct.	Sorting algorithm is incorrect or not bubble sort implementation.
Sort is implemented in function <code>string_compare</code> .	String compare logic implemented in separate function.	String compare logic implemented in main function.	No logic for implementation of String compare.

the code. We term these instructions as *coding criteria*. These are given by instructors primarily so that a specific learning goal of programming can be evaluated; e.g., in the problem in the Figure 1, the instructor wants students to implement *bubble-sort*. There may also be common code quality criteria such as the algorithm complexity, modularity (use of functions), variable names, or comments, which may not be written explicitly in the problem statement, but appear in the evaluation guidelines.

Any such general assessment of code quality or checking of adherence to specific coding criteria has to be done *manually* by teaching assistants (TAs). E.g. for the problem in Figure 1, if a student submitted a program implementing a sorting algorithm other than bubble-sort, a TA would have to nullify the testcase marks and possibly award a zero. Furthermore, the TAs would inspect the source code of non-working programs, i.e. programs that fail the testcase check. The instructor’s grading policy may allow for a generous award of partial marks if the TA can fix a small error and get the program to work.

Thus, a *source code inspection* based assessment of submitted program is a must, in an educational setting. The most systematic way to perform such grading is using a *rubric* [30]. In Table 1, we present a rubric for the problem statement shown in Figure 1. A rubric has multiple *criteria* (in rows), where each criterion has a range of *ratings*. The ratings are ordered from highest (meets the criterion fully) to lowest (does not meet the criterion at all). TAs are expected to select one of the ratings, and optionally explain why those ratings were chosen. The ratings typically correspond to some marks and are added up to arrive at the total code quality and criteria-based marks. How these marks combine with the testcase marks is a policy that may vary with the instructor. The Bodhitree-Evalpro interface, used in this work presents a convenient rubric-based grading interface to a Grader TA, as shown in Figure 2. The student’s source code is shown alongside the rubric so that the TA can easily check the various criteria and select appropriate ratings.

3 Related Work

Automated grading of programming assignments in computer science education has gained increased interest over the past few decades. Several studies [4, 15, 21, 33] highlight that automated grading systems benefit students and reduce instructors grading efforts. There are numerous existing tools that provide the testcase based

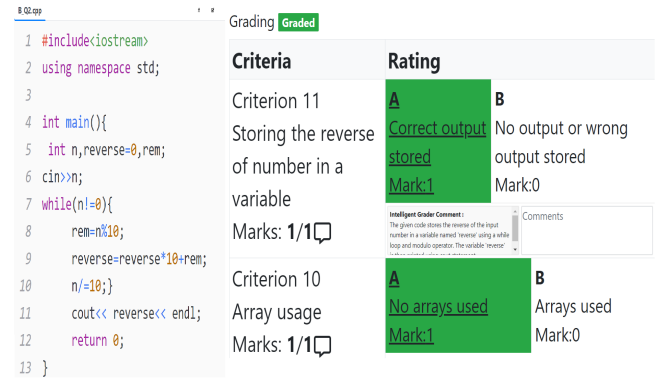


Figure 2: Grading Interface of BodhiTree-EvalPro, showing student submitted code in the left pane and the Rubric with criteria and ratings in the right pane

automatic grading and feedback to the students [2, 12, 23, 25, 31]. However, all these tools grade only those submissions that compile.

Kelly *et. al.* [15] uses straightforward pattern matching of test-cases outputs to provide partial marks for mismatches due to trailing or excess spaces, incorrect sequence, and case sensitivity. However, test cases that can truly reveal some partial ‘logical incorrectness’ (as opposed to output formatting issues) by exercising corner cases, requires expertise in designing such test cases. Even then, test case based evaluation does not allow for partial grading when the code is non-compileable, or completely failing in its runs. Thus, approaches have been developed that directly evaluate the source code. Some approaches consider the similarity and semantic analysis of student code compared to reference solutions. Xiao Liu *et al.* [18] present the AUTOGRADER tool, which assesses the correctness of the student code by finding the semantically different execution paths between the student code and the reference solution using symbolic execution, and also provides counter examples as feedback based on path deviations. Another work Gradeer [10] is an auto-grading tool for Java programs that incorporates test case checks, similarity checks, and manual grading. However, these similarity-based approaches cannot analyze non-compiled code and heavily depend on reference solution(s). Additionally, large and complex set of rules have to be developed, which may not generalise to diverse programs.

Recently there has been significant amount of work utilizing machine learning for code quality based grading. Most of the existing machine learning approaches [22, 29, 34] captures the syntax or semantic features extracted from the abstract representations of a given student code. Srikant *et. al.* [28, 29] proposed an approach where the system grades how similar the programming logic to the correct solution based on a rubric using a machine learning approach based on highly informative features derived from abstract representation of a program. In this approach, the challenge is that it needs large amount of feature-related data to train the model for each question.

Parihar *et. al.* [22] presents a framework GradeIT, which is introduced as a solution for automated grading and program repairing in introductory programming courses. Their framework utilizes program repair to grade submissions that do not compile due to minor syntax errors, allowing for partial marks. This framework

uses some parts of the test cases as a feature to provide input to the machine learning model using the linear regression algorithm. This approach primarily uses test case evaluations. It does not check any specific subjective qualities given in the problem statement.

One of the prior works that is similar to the criterion-based grading is that of Chourasia *et al.* [9] which uses transfer learning to identify sorting, searching and path finding algorithms in source code. It treats them as a classification task with deep neural network models such as CodeBERT and GraphCodeBERT. A key limitation is the need for large training datasets for each type of algorithm, and a separate model per problem statement, which does not generalize well to other functionalities.

Marcus Messer, *et al.* in their study [20] mention that research with LLMs in computer science education has increased. Some works [5] describe LLM-based code generation models used in educational settings for auto-grading and assignment generation. Recent research [27] indicates that programming exercise generation and code explanation are promising applications of GPT in programming education.

To address various challenges of code evaluation, our work employs LLMs to evaluate both the correctness and quality of code as specified by the instructor. We introduce a tool called **TA-Buddy**, which integrates this LLM-based model to assist a TA in grading student code according to the instructor’s rubric. To our knowledge this is the first tool with such generalized capabilities to evaluate source code and generate suggested grades against a rubric.

4 AI-Assisted Rubric-Based Grading with TA-Buddy

While our Bodhitree-Evalpro interface already offered a fully manual interface to grade a source code with reference to a rubric, it now also offers an *AI-Assisted manual grading* option, which we term ‘TA-Buddy’ in this paper. Figure 3 shows the sequence in which AI-Assisted grading is meant to be used. First, the instructor (or a senior TA) configures the problem statement, the testcases, and the source code grading rubrics. The problem is then released to the students (with rubrics and some visible testcases). Students submit their assignments up to some deadline. After the deadline, grading can begin. First, the instructor goes to the submissions interface and runs the conventional testcase evaluation for all the submissions. Then the instructor can choose to auto-grade the source code using the AI TA-BUDDY by clicking one button. The AI-powered TA-BUDDY will choose the rating for each criterion and return them along with AI-generated reasoning. Grader TAs can see the suggested ratings marked by an underline (e.g. in Figure 2, rating A is underlined for both criteria, so it is the one suggested by the TA Buddy). The associated generated reasoning is seen in the ‘Intelligent Grader’ Comments section. Human graders can either confirm the pre-populated grades or overrule them by selecting another option and providing a comment if required.

5 AI Model for Selection of Ratings in the Grading Rubric

Our AI autograder, TA-BUDDY, is powered by a pre-trained large language model. We use *Code Llama* [26] as a base model and fine-tune it with a training dataset of problem statements, rubrics,

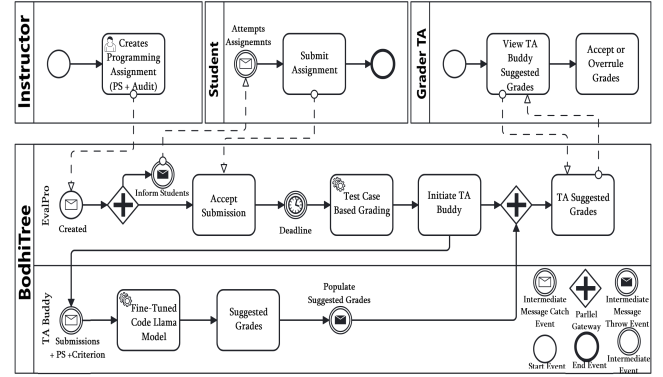


Figure 3: Workflow for AI-Assisted Manual Grading of Source Code

student codes and selected ratings for the rubric criteria for each student code. Code Llama is a family of open-source LLMs for code, built on Llama 2 [32], a pre-trained and fine-tuned family of LLMs. Code Llama includes models with 7B, 13B, 34B, and 70B parameters, which are capable of tasks such as code generation and code infilling, and support large input context lengths.

We began our research on source code grading in mid-2023, when CodeLlama-7B was released. CodeLlama-7B appeared to be one of the better performing models at that time [17] in code generation tasks. Furthermore it met our requirements that the model we use should be open source and should run on one GPU with 48GB memory. CodeLlama showed further improved performance (15% better accuracy over few-shot prompting) with fine-tuning methods. Thus, we build our grader model by fine-tuning the Code Llama-7b-Instruct model with the Direct Preference Optimization (DPO) method [24]. For our grading task, the input is

- The programming problem statement,
- Student submission,
- The instructor specified rubric.

The Code Llama model is asked, through a carefully designed prompt format [6], to provide as output, the most appropriate rating for each criterion in the provided rubric for student-submitted programs.

5.1 Training data

While CS101 has been taught for the past many decades in IIT Bombay (and many other colleges around the world) with subjective grading of source code, data regarding such marks and the rubrics used was not available. What we did have access to though was problem statements and submissions from various past CS101 offerings in IIT Bombay. To create the training dataset, we collected this data from 21 C++ programming assignments (problems) from CS101 course offerings of the past 2 years, with a total of 2914 submissions and 155 criteria. Each assignment was part of a proctored graded test given to students during the course, where students worked on a machine in a lab in the department with no network access. Thus the source code is uninfluenced by any internet sources, and there is also no danger of plagiarism from friends’ submissions.

Rubrics were created for each of these problems, by this author team which includes current and past CS101 instructors and TAs. The criteria in the rubrics were based directly on the original

instructor-specified criteria in the problem statements. However, the *partial marking* policies expressed in the form of *ratings* of the rubric criteria were the decision only of the author team of this paper.

Following the creation of these rubrics, we conducted a manual grading exercise with around 24 TAs, who graded anonymized student submissions according to the respective rubrics. The graded student codes were then collected, and bundled with the corresponding problem statements and rubrics with respective ratings to form the dataset required for fine-tuning Code Llama.

The runs for fine-tuning Code Llama-7B-Instruct model were carried out on NVIDIA A100 SXM4-80GB using a single GPU and took 390 minutes. This fine-tuned classifier model is integrated into the Bodhitree-Evalpro interface to grade a student submission with a grading rubric.

6 Evaluation of AI-Assisted Grader - TA Buddy

Our AI grader assistant, TA-Buddy was evaluated for its usefulness as compared with ‘pure’ manual grading. The main questions we wanted to answer about TA-Buddy are as follows:

- (1) What is the ‘acceptance rate’ by the human TAs of the AI TA-Buddy’s suggested grades?
- (2) Overall, taking into account time spent in verifying and possibly over-ruling the AI-suggested grades, does TA-Buddy save the total person-minutes spent in manual grading? If so, how much time does it save?
- (3) How different are the ratings selected by the AI-assisted human grader as compared to those selected by the purely manual human grader?

6.1 Experiment Design

Our experiment design for answering these questions was as follows. We again used problem statements (*different from those used for training*) from past CS101 offerings, and created detailed rubrics that captured any source code guidelines that the instructor had laid out. A total of 6 programming problems of varying difficulty levels were selected with a total of 740 submissions (Table 2). Conventional testcases and source code grading rubrics were configured, with a total of 55 criteria. Each problem’s rubric included some common criteria as well as some criteria unique to the problem. E.g. the rubric of the Problem 2 of *reversing a positive integer* included unique criteria such as: (1) “Is each digit of the original number correctly extracted?”, (2) “Are digits being correctly updated and utilised to calculate the number’s reverse?” and (3) “Is the loop iterated correctly for updating all digits of the number to the final number?”.

This problem set was configured once each in *two separate courses* in our Bodhitree-Evalpro interface. The 740 submissions were also uploaded into each course. Thus we had two copies of the entire evaluation dataset. One course (C1) was used for pure manual grading, and another course (C2) was used for AI-assisted grading. In both C1 and C2, we first ran the conventional testcase autograding. In C2, the Code Llama-powered TA-BUDDY was used for first obtaining suggested ratings for all the criteria for all the submissions. We recruited fifteen TAs (graduate students, some of whom had been past CS 101 TAs) to grade the submissions in courses

Table 2: Summary Statistics of the Evaluation Dataset. For details see [6]

Problem	Problem Complexity	No. of Submissions	No. of Criteria	Avg No. of Lines
#1: Evaluating Expressions	Easy	150	6	11
#2: Reversing Numbers	Easy	150	11	27
#3: Binary To Decimal	Moderate	70	12	31
#4: Complex Number Operations	Difficult	150	14	48
#5: Sorting Strings	Difficult	70	6	32
#6: Strictly Increasing/Decreasing	Moderate	150	6	17

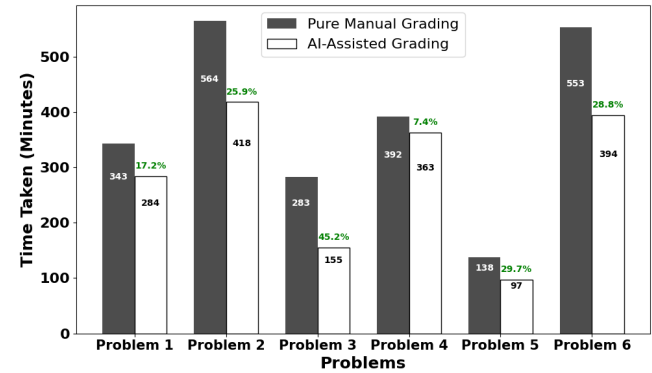


Figure 4: Comparison of Time required by Grader TAs. Pure Manual Grading Vs AI-Assisted Manual Grading

C1 as well as C2. Thus each submission was graded *twice*, once in a ‘pure’ manual manner by a Grader TA in C1, and once by a *different* Grader TA who could see the AI-Assistant TA-Buddy’s suggested grades in C2 and either confirmed or over-ruled those grades. Each TA graded roughly the same number of submissions. The session was conducted under the supervision of our R&D team who ensured that timings for doing this work were meticulously maintained.

Thus the data required to answer the first three questions is readily available from this setup.

6.2 Results

As shown in Figure 4, compared with pure manual grading time, AI-assisted grading was observed to require less time for all problems, by a range of 7.40% to 45.23%. Note that the AI-assisted grading time includes time taken by a TA to inspect the student source code, verify whether the AI-generated grade is correct, possibly select another grade and write a different comment.

Since the AI-Assisted grading is done after the AI grader run, it is important to measure the time taken for that run. Table 3 shows the time taken by TA Buddy to generate suggested grades. A single inferencing query took 6-10 seconds on our platform. AI grading runs do take a non-trivial amount of time, however, it is worth noting that these runs are simply triggered by the instructor with a button click, with no further no human intervention. The experiment we did generates *three* different grades for each submission:

- (1) the *Pure Manual Grade* (PMG), which is the one selected by the Grader TA who is grading in course C1 - i.e. with no AI-Suggested Grades (AISG).
- (2) the *AI-Suggested Grade* that is visible to the Grader TA in course C2 after the instructor does an auto-grading run

Table 3: AI Grader (TA Buddy) Run Time

	No. of Submissions	No. of Criteria	Total AI Grading Time (In Mins)	Avg. AI Grading Time Per Submission Per Criterion (in Secs)
#1	150	6	95	6.33
#2	150	11	158	5.74
#3	70	12	107	7.64
#4	150	14	232	6.62
#5	70	6	72	10.2
#6	150	6	150	10

Table 4: AI-Suggested Grades (AISG) compared with AI-Assisted Grades (AIAG) and Pure Manual Grades (PMG)

	AIAG = AISG	AIAG = PMG
#1	79.81%	91.95%
#2	88.48%	90.31%
#3	85.85%	91.99%
#4	84.82%	94.64%
#5	60.55%	87.38%
#6	67.91%	87.12%
Average	77.57%	90.55%

- (3) the *AI-Assisted Grade* (AIAG) that is the final grade entered by the Grader TA in course C2, after either confirming or over-ruling the AI-suggested grade.

Table 4 shows two comparisons: first, of the AI-suggested grade with the final AI-assisted grade and second, the AI Assisted Grade with the Pure Manual Grade, for six problems (Problem #1 to Problem #6). The “AIAG=AISG” percentage is calculated as follows: for each criterion of the problem, we calculate the percentage of submissions where the ratings selected by a TA with AI assistance exactly matched the AI suggested ones. The table shows the average across all the criteria for that problem. This measures how helpful the TA Buddy appeared to TAs who graded in course C2. The “AIAG=PMG” percentage is similarly defined by comparing the ratings by a TA with AI assistance with those selected by a TA without any AI assistance. This measures whether or how much the TAs are influenced by the visibility of the AI-suggested grades. Finally the last row shows the average match percentage over these six problems. Grader TAs seem to agree with the TA Buddy suggested grade only 60% of the time in the worst case, but 88% of the time in the best case. However, the final grade selected by the AI-Assisted grader TA agrees with the ‘pure manual grade’ on an average 90% of the time, which confirms that the suggested grades are not unduly influencing the Grader TA’s final choice of grades.

7 Discussion and Conclusions

In this paper, we presented an LLM-based grading tool *TA Buddy*, for introductory programming assignments, and its experience of usage in a grading experiment on past lab problems of the CS101 course. Our results indicate that the tool on an average saved grading time by 24% with an average grading accuracy of 78% compared to the final (AI-assisted) grade chosen by a human grader.

In contrast to the existing autograding approaches, such as test-case-based grading or similarity-based grading methods which do not cover all the aspects of code quality checking, our grading system *TA Buddy* offers effective assistance for speedy and accurate checking of generalized source code quality and criteria by human TAs. This can free TAs using our Bodhitree-Evalpro interface to focus more on addressing student doubts and directly helping students improve their programming capabilities.

Our tool facilitates convenient awarding of partial marks for student submissions. In our study, involving 6 lab assignments, our rubric-based grading approach resulted in improved marks for over 50% student submissions that contained compilation errors when compared to traditional testcase-based grading methods, for an average increase of marks by 16%.

Compared to the reference code-based solutions, our solution leverages extensive pre-training knowledge of LLMs, eliminating the need for sample solutions and extensive rule generation for grading. This capacity allows it to handle various program complexities and concept variations effectively.

Some drawbacks and thus scope for future work are as follows. Our AI Grader model faces increased processing time as the number of criteria and code length grow. This increase occurs regardless of problem difficulty due to longer inference times for more complex inputs.

Another limitation of the fine-tuned model is that the training dataset is somewhat skewed in that a large number of submissions seem to get high ratings on many of the criteria. We conjecture that this may be due to the fact that all our data is from lab exams, which students write seriously, and after adequate preparation.

The base LLM model, Code Llama, has some inherent biases due to data it is trained on. Which contains the frequently used built-in functions like *sqrt*, *sin*, and *pow* along with conventional data structures. In our study, we observed that for problem 5 in Table 2, which involves sorting strings using selection sort, the model compared the sort function with the built-in string sort function, due to which it gives incorrect ratings for the criterion *sorting algorithm implementation*. These limitations highlight the importance of providing clear, explicit rubric criteria that specifically address the expected implementation approach. In the future we plan to carry out a systematic study of such limitations and improve the accuracy of the model.

Additionally, during the experiment discussed in Section 6, some of the TAs struggled with the AI-suggested ratings because of inaccuracies in ratings and reasoning. For them, correcting these wrong ratings and reasonings lead to increased grading time. We are planning to carry out a formal feedback survey of the grader TAs, so as to gauge the experience of the TAs which can give us different insights than what the grading accuracy and grading time numbers can provide.

Lastly, since our experiments were (intentionally) on past CS 101 submissions, we were not able to carry out a learner survey. We will also soon be using our Bodhitree-Evalpro interface with *TA Buddy* in a real upcoming CS 101 offering in our college. As a result we will study whether there is any indirect benefit of *TA Buddy* to learners.

Acknowledgments

This research was a team effort, and we would like to thank Sainath V, Nithin CGS, Kaushikraj Maharajan, and Jayant Havare for their crucial roles in building the fine tuning model. We also appreciate D. Sandeep Varma and Ch. Vinay Kumar for their support with the data collection interface. Special thanks to Firuza Karmali, Nageshrau Karmali, and the entire Bodhitree team for their technical advice and support during the field trials of our tool for this case study.

References

- [1] Hussam Aldriye, Asma Alkhalaf, and Muath Alkhalaf. 2019. Automated grading systems for programming assignments: A literature review. *International Journal of Advanced Computer Science and Applications* 10 (2019), 3.
- [2] Autolab Project. 2020. Autolab Project. <http://www.autolabproject.com>
- [3] M Rifky I Bariansyah, Satrio Adi Rukmono, and Riza Satria Perdana. 2021. Semantic approach for increasing test case coverage in automated grading of programming exercise. In *2021 International Conference on Data and Software Engineering (ICoDSE)*. 1–6.
- [4] Yun-Zhan Cai and Meng-Hsun Tsai. 2019. Improving programming education quality with automatic grading system. In *Proceedings of Innovative Technologies and Learning: Second International Conference, ICITL 2019, Tromsø, Norway*. Springer, Norway, 207–215.
- [5] Doga Cambaz and Xiaoling Zhang. 2024. Use of AI-driven Code Generation Models in Teaching and Learning Programming: a Systematic Literature Review. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*. Association for Computing Machinery, New York, NY, United States, 172–178.
- [6] Saurav Chaudhary. 2024. *AI-Driven ML Pipeline for Subjective Grading on EvalPro (Stage I Report)*. Master's thesis. Indian Institute of Technology, Bombay. <https://github.com/sauravchaudharysc/MTP-Stage-1/>.
- [7] Kameswari Chebrolu. 2024. BodhiTree, An E-Learning Academy. <https://wiki.bodhi-tree.in/>.
- [8] Chih-Yueh Chou and Yan-Jhih Chen. 2021. Virtual teaching assistant for grading programming assignments: Non-dichotomous pattern based program output matching and partial grading approach. In *2021 IEEE 4th International Conference on Knowledge Innovation and Invention (ICKII)*. IEEE, Taiwan, 170–175.
- [9] Pranshu Chourasia, Ganesh Ramakrishnan, Varsha Apte, and Suraj Kumar. 2022. Algorithm identification in programming assignments. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*. Milan Italy, 471–481.
- [10] Benjamin Clegg, Maria-Cruz Villa-Urriol, Phil McMinn, and Gordon Fraser. 2021. Gradeer: an open-source modular hybrid grader. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*. 60–65.
- [11] Christopher Douce, David Livingstone, and James Orwell. 2005. Automatic test-based assessment of programming: A review. *Journal on Educational Resources in Computing (JERIC)* 5, 3 (2005), 4–es.
- [12] Stephen H Edwards and Manuel A Perez-Quinones. 2008. Web-CAT: automatically grading programming assignments. In *Proceedings of the 13th annual conference on Innovation and technology in computer science education*. ACM New York, NY, USA, 328–328.
- [13] Cristian D González-Carrillo, Felipe Restrepo-Calle, Jhon J Ramírez-Echeverry, and Fabio A González. 2021. Automatic grading tool for jupyter notebooks in artificial intelligence courses. *Sustainability* 13, 21 (2021), 12050.
- [14] Sumit Gulwani, Ivan Radiček, and Florian Zuleger. 2018. Automated clustering and program repair for introductory programming assignments. *ACM SIGPLAN Notices* 53, 4 (2018), 465–480.
- [15] Emlyn Hegarty-Kelly and Dr Aidan Mooney. 2021. Analysis of an automatic grading system within first year computer science programming modules. In *Proceedings of the 5th Conference on Computing Education Practice*. 17–20.
- [16] Jagpreet. 2024. The Rise and Benefits of MOOCs Landscape. <https://www.learningroutes.in/blog/rise-and-benefits-of-moocs-landscape> Accessed: July 2024.
- [17] Jeremy K. 2023. Code Llama: quick start guide and prompt engineering. <https://medium.com/aimonks/code-llama-quick-start-guide-and-prompt-engineering-eb1de8758399>.
- [18] Xiao Liu, Shuai Wang, Pei Wang, and Dinghao Wu. 2019. Automatic grading of programming assignments: an approach based on formal semantics. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*. IEEE Press, Montreal Quebec Canada, 126–137.
- [19] Marcus Messer, Neil CC Brown, Michael Kölling, and Miaoqing Shi. 2023. Machine learning-based automated grading and feedback tools for programming: A meta-analysis. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1*. 491–497.
- [20] Marcus Messer, Neil CC Brown, Michael Kölling, and Miaoqing Shi. 2024. Automated grading and feedback tools for programming education: A systematic review. *ACM Transactions on Computing Education* 24, 1 (2024), 1–43.
- [21] Joydeep Mitra. 2023. Studying the impact of auto-graders giving immediate feedback in programming assignments. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*. 388–394.
- [22] Sagar Parihar, Ziyaan Dadachanji, Praveen Kumar Singh, Rajdeep Das, Amey Karkare, and Arnab Bhattacharya. 2017. Automatic grading and feedback using program repair for introductory programming courses. In *Proceedings of the 2017 ACM Conference on Innovation and Technology in Computer Science Education*. Association for Computing Machinery, New York, NY, United States, 92–97. <https://doi.org/10.1145/3059009.3059026>
- [23] Pieter Abbeel, Arjun Singh, Ibrahim Awwal, Sergey Karayev. 2014. GradeScope. <https://www.gradescope.com/>.
- [24] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2024. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems* 36 (2024), 1–35.
- [25] Replit. 2020. repl.it classroom. <https://repl.it/site/classrooms> Accessed: 2019-06-11.
- [26] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* 3 (2023), 48.
- [27] Jaromir Savelka, Arav Agarwal, Christopher Bogart, Yifan Song, and Majd Sakr. 2023. Can generative pre-trained transformers (gpt) pass assessments in higher education programming courses?. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1*. 117–123.
- [28] Shashank Srikant and Varun Aggarwal. 2013. Automatic grading of computer programs: A machine learning approach. In *2013 12th International Conference on Machine Learning and Applications*. 85–92.
- [29] Shashank Srikant and Varun Aggarwal. 2014. A system to grade computer programming skills using machine learning. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. Association for Computing Machinery, New York, NY, United States, 1887–1896.
- [30] Martijn Stegeman, Erik Barendsen, and Sjaak Smetsers. 2016. Designing a rubric for feedback on code quality in programming courses. In *Proceedings of the 16th Koli Calling International Conference on Computing Education Research*. 160–164.
- [31] Stepik. 2020. Stepik - smart tools for IT instructors. <https://stepik.org/catalog> Accessed: 2020-07-11.
- [32] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shriti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models.
- [33] Zahid Ullah, Adidah Lajis, Mona Jamjoom, Abdulrahman Altalhi, Abdullah Al-Ghamdi, and Farrukh Saleem. 2018. The effect of automatic assessment on novice programming: Strengths and limitations of existing systems. *Computer Applications in Engineering Education* 26, 6 (2018), 2328–2341.
- [34] Arjun Verma, Prateksha Udhayan, Rahul Murali Shankar, Nikhila Kn, and Sujit Kumar Chakrabarti. 2021. Source-code similarity measurement: syntax tree fingerprinting for automated evaluation. In *Proceedings of the First International Conference on AI-ML Systems*. Association for Computing Machinery, New York, NY, United States, 1–7. <https://doi.org/10.1145/3486001.3486228>
- [35] Susilo Veri Yulianto and Inggriani Liem. 2014. Automatic grader for programming assignment using source code analyzer. In *2014 International Conference on Data and Software Engineering (ICODSE)*. 1–4. <https://doi.org/10.1109/ICODSE.2014.7062687>